

ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

ĐỒ ÁN TỐT NGHIỆP

**Nền tảng chia sẻ ảnh thông minh tích hợp kiểm duyệt
và tìm kiếm ngữ nghĩa**

HOÀNG CHÍ THANH

thanh.hc225929@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: _____

Khoa: _____

Trường: Công nghệ thông tin và Truyền thông

ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

ĐỒ ÁN TỐT NGHIỆP

**Nền tảng chia sẻ ảnh thông minh tích hợp kiểm duyệt
và tìm kiếm ngữ nghĩa**

HOÀNG CHÍ THANH

thanh.hc225929@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: _____

Khoa: _____

Trường: Công nghệ thông tin và Truyền thông

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn cùng các thầy cô thuộc Viện Công nghệ Thông tin và Truyền thông, Trường Đại học Bách khoa Hà Nội đã định hướng và trang bị những kiến thức chuyên môn quý báu trong suốt quá trình học tập.

Đồng thời, em xin bày tỏ lòng biết ơn sâu sắc đến gia đình, người yêu và bạn bè đã luôn là chỗ dựa tinh thần, động viên và hỗ trợ mọi mặt để em yên tâm hoàn thành đồ án này.

TÓM TẮT

Sự bùng nổ của nghệ thuật số kéo theo nhu cầu lớn về các nền tảng chia sẻ ảnh trực tuyến chuyên biệt. Tuy nhiên, các hệ thống hiện tại thường gặp khó khăn trong việc kiểm duyệt nội dung độc hại trên quy mô lớn, ngăn chặn vấn nạn sao chép tác phẩm, và cung cấp công cụ tìm kiếm ngữ nghĩa thông minh thay vì chỉ dựa vào thẻ băm truyền thống. Bên cạnh đó, việc xử lý các tác vụ tính toán nặng ngay trên luồng yêu cầu chính dễ gây nghẽn hệ thống và làm giảm trải nghiệm người dùng. Để giải quyết các hạn chế này, đề án lựa chọn hướng tiếp cận xây dựng hệ thống theo kiến trúc module hóa kết hợp với cơ chế xử lý bất đồng bộ qua hàng đợi nền nhằm tối ưu hiệu năng và đảm bảo khả năng mở rộng linh hoạt. Trên cơ sở đó, tác giả đã phát triển một nền tảng chia sẻ ảnh toàn diện sử dụng Next.js cho phía giao diện và NestJS cho phía dịch vụ, lưu trữ dữ liệu trên PostgreSQL và tối ưu tìm kiếm toàn văn bằng Meilisearch. Hệ thống tích hợp Clerk để xác thực người dùng và Stripe để xử lý mô hình thành viên trả phí cho nhà sáng tạo. Điểm nhấn công nghệ của giải pháp là việc ứng dụng mô hình CLIP sinh vector embedding phục vụ tìm kiếm ngữ nghĩa qua pgvector, sử dụng thuật toán Perceptual Hashing (pHash) phát hiện ảnh trùng lặp, và tích hợp NSFWJS tự động phân loại nội dung nhạy cảm. Toàn bộ các tiến trình nặng này được điều phối bất đồng bộ bởi BullMQ và Redis để giải phóng luồng xử lý chính. Đóng góp chính của đề án là đề xuất và hiện thực hóa thành công một kiến trúc phân tán thực tế, giải quyết hài hòa giữa trải nghiệm người dùng, quyền lợi kinh tế của nhà sáng tạo và hiệu quả quản trị tự động. Kết quả sau cùng đạt được là một khung hệ thống hoàn chỉnh, vận hành ổn định với độ trễ thấp và đáp ứng tốt các yêu cầu vận hành thực tế.

Từ khóa: chia sẻ ảnh, NestJS, Next.js, pHash, CLIP, pgvector, Meilisearch, BullMQ, Stripe.

ABSTRACT

The boom of digital art has driven a massive demand for specialized online image-sharing platforms. However, existing systems often struggle to moderate harmful content at scale, prevent artwork plagiarism, and provide intelligent semantic search instead of relying solely on traditional hashtags. Furthermore, executing computationally heavy tasks directly on the main request path can easily cause system bottlenecks and degrade user experience. To address these limitations, this thesis adopts an approach that constructs the system based on a modular architecture combined with an asynchronous background queue mechanism to optimize performance and ensure flexible scalability. On this basis, a comprehensive image-sharing platform has been developed using Next.js for the frontend and NestJS for backend services, utilizing PostgreSQL as the relational database and Meilisearch for optimized full-text search. The system integrates Clerk for authentication and Stripe to handle paid membership models for creators. The technological highlight of this solution lies in applying the CLIP model to generate vector embeddings for semantic search via `pgvector`, utilizing the Perceptual Hashing (pHash) algorithm for duplicate image detection, and integrating NSFWJS for automated sensitive content classification. All of these intensive processes are coordinated asynchronously by BullMQ and Redis to free up the main execution path. The main contribution of this thesis is the proposal and successful implementation of a practical distributed architecture that harmoniously balances user experience, financial rights for creators, and automated administration efficiency. The final outcome is a complete system skeleton that operates stably with low latency and effectively meets real-world operational requirements.

Keywords: image sharing, NestJS, Next.js, pHash, CLIP, `pgvector`, Meilisearch, BullMQ, Stripe.

MỤC LỤC

LỜI CẢM ƠN	i
TÓM TẮT	ii
ABSTRACT	iii
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	3
1.3 Định hướng giải pháp.....	4
1.4 Bố cục đồ án	6
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	8
2.1 Khảo sát hiện trạng	8
2.1.1 Các nền tảng chia sẻ ảnh và cộng đồng nghệ thuật số	8
2.1.2 Quy trình kiểm duyệt nội dung trên các nền tảng hiện có	8
2.1.3 Cơ chế tìm kiếm và khám phá nội dung.....	8
2.1.4 Mô hình kiểm tiền của người sáng tạo	8
2.1.5 Tổng kết hạn chế	8
2.2 Tổng quan tác nhân và chức năng	8
2.2.1 Tác nhân Khách	8
2.2.2 Tác nhân Người dùng thường	8
2.2.3 Tác nhân Người sáng tạo.....	8
2.2.4 Tác nhân Quản trị viên và Điều phối viên.....	8
2.2.5 Biểu đồ use case tổng quan	8
2.3 Đặc tả yêu cầu chức năng	8
2.3.1 Đăng ký, đăng nhập và đồng bộ tài khoản.....	8
2.3.2 Đăng tải và quản lý tác phẩm	8
2.3.3 Kiểm duyệt và xử lý báo cáo	8
2.3.4 Tìm kiếm, khám phá và tương tác	8
2.3.5 Quản lý gói thành viên và thanh toán	8

2.4 Yêu cầu phi chức năng	8
2.4.1 Hiệu năng và khả năng đáp ứng	8
2.4.2 An toàn, bảo mật và phân quyền	8
2.4.3 Độ tin cậy của xử lý nền	8
2.4.4 Khả năng mở rộng và bảo trì	8
2.4.5 Khả năng quan sát và truy vết.....	8
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	9
3.1 Nền tảng Backend và cơ sở dữ liệu	9
3.1.1 Kiến trúc NestJS Framework.....	9
3.1.2 Prisma ORM	10
3.1.3 PostgreSQL và mô hình dữ liệu quan hệ.....	11
3.2 Nền tảng Frontend và State Management	12
3.2.1 Next.js App Router.....	12
3.2.2 Mantine UI.....	13
3.2.3 Zustand và phân loại trạng thái frontend.....	14
3.3 Lý thuyết về trí tuệ nhân tạo và thuật toán	14
3.3.1 Mô hình CLIP và học biểu diễn đa phương thức.....	14
3.3.2 Mô hình NSFWJS và phân loại hình ảnh	16
3.3.3 Thuật toán pHash và khoảng cách Hamming	17
3.3.4 Không gian vector, KNN và pgvector.....	19
3.4 Các dịch vụ trung gian và hạ tầng xử lý	21
3.4.1 Message Queue với BullMQ và Redis	21
3.4.2 Meilisearch và inverted index	21
3.4.3 Clerk Auth.....	22
3.4.4 Stripe và quản lý thanh toán	23
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ	
HỆ THỐNG	24
4.1 Thiết kế kiến trúc hệ thống	25
4.1.1 Lựa chọn kiến trúc phần mềm	25
4.1.2 Kiến trúc tổng thể	25
4.1.3 Thiết kế các module backend	25
4.1.4 Thiết kế các phân vùng frontend.....	25

4.1.5 Thiết kế luồng xử lý nền	25
4.2 Thiết kế cơ sở dữ liệu	25
4.2.1 Mô hình thực thể quan hệ.....	25
4.2.2 Thiết kế bảng người dùng và phân quyền.....	25
4.2.3 Thiết kế bảng tác phẩm và ảnh	25
4.2.4 Thiết kế bảng tương tác xã hội.....	25
4.2.5 Thiết kế bảng thanh toán và gói thành viên	25
4.3 Thiết kế API và giao tiếp hệ thống	25
4.3.1 API xác thực và người dùng	25
4.3.2 API quản lý tác phẩm	25
4.3.3 API tìm kiếm và khám phá.....	25
4.3.4 API quản trị và kiểm duyệt.....	25
4.3.5 API thanh toán và webhook	25
4.4 Triển khai hệ thống.....	25
4.4.1 Môi trường phát triển	25
4.4.2 Cấu hình cơ sở dữ liệu và dịch vụ trung gian.....	25
4.4.3 Triển khai frontend.....	25
4.4.4 Triển khai backend.....	25
4.4.5 Triển khai worker xử lý nền.....	25
4.5 Đánh giá và kiểm thử	25
4.5.1 Kiểm thử chức năng	25
4.5.2 Kiểm thử tích hợp	25
4.5.3 Đánh giá hiệu năng xử lý ảnh	25
4.5.4 Đánh giá tìm kiếm.....	25
4.5.5 Đánh giá bảo mật và phân quyền	25
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT.....	26
5.1 Tự động hóa pipeline xử lý ảnh	26
5.1.1 Tách tác vụ nặng khỏi luồng request	26
5.1.2 Quản lý tiến độ và trạng thái xử lý	26
5.1.3 Xử lý lỗi cục bộ ở cấp ảnh.....	26
5.2 Kiểm duyệt nội dung bán tự động.....	26
5.2.1 Phân loại NSFW trong worker.....	26

5.2.2 Lưu metadata phục vụ điều phối.....	26
5.2.3 Cân bằng giữa tự động hóa và kiểm duyệt thủ công	26
5.3 Phát hiện ảnh trùng lặp bằng pHash.....	26
5.3.1 Sinh dấu vân tay nhận thức.....	26
5.3.2 So sánh bằng khoảng cách Hamming	26
5.3.3 Ứng dụng trong bảo vệ nội dung.....	26
5.4 Tìm kiếm lai toàn văn và ngữ nghĩa.....	26
5.4.1 Lập chỉ mục Meiliseach.....	26
5.4.2 Lưu trữ vector bằng pgvector.....	26
5.4.3 Kết hợp kết quả tìm kiếm trong trải nghiệm người dùng.....	26
5.5 Mô hình thành viên cho người sáng tạo.....	26
5.5.1 Thiết kế artist tier	26
5.5.2 Tích hợp Stripe subscription.....	26
5.5.3 Kiểm soát quyền truy cập nội dung	26
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	27
6.1 Kết luận	27
6.1.1 Kết quả đạt được	27
6.1.2 Mức độ đáp ứng mục tiêu đề tài.....	27
6.1.3 Các hạn chế còn tồn tại	27
6.2 Hướng phát triển.....	27
6.2.1 Mở rộng khả năng kiểm duyệt	27
6.2.2 Tối ưu tìm kiếm vector.....	27
6.2.3 Cải tiến hệ thống khuyến nghị	27
6.2.4 Mở rộng cơ chế bản quyền và khiếu nại.....	27
6.2.5 Tối ưu triển khai hạ tầng quy mô lớn.....	27
TÀI LIỆU THAM KHẢO.....	28

DANH MỤC HÌNH VẼ

DANH MỤC BẢNG BIỂU

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong khoảng hơn một thập niên gần đây, các cộng đồng chia sẻ ảnh và nghệ thuật số đã phát triển từ những không gian lưu trữ tác phẩm đơn giản thành các hệ sinh thái xã hội có quy mô lớn, nơi người dùng vừa tiêu thụ nội dung, vừa tương tác, theo dõi nghệ sĩ, xây dựng bộ sưu tập cá nhân và tham gia vào các mô hình kinh tế sáng tạo. Những nền tảng như vậy không còn chỉ phục vụ nhu cầu đăng tải ảnh tĩnh, mà đã trở thành hạ tầng trung gian giữa tác giả, người xem, cộng đồng người hâm mộ và các bên điều phối nội dung. Đối với lĩnh vực minh họa số, manga, concept art và nội dung sáng tạo bằng trí tuệ nhân tạo, tốc độ sinh nội dung ngày càng cao làm cho bài toán vận hành nền tảng trở nên phức tạp hơn đáng kể so với các hệ thống quản lý ảnh truyền thống.

Điểm đặc thù của một nền tảng chia sẻ ảnh là mỗi đơn vị dữ liệu không chỉ có tiêu đề, mô tả và thẻ gắn kèm, mà còn mang nội dung thị giác có dung lượng lớn, nhiều biến thể độ phân giải, nhiều trạng thái xử lý và nhiều rủi ro về bản quyền, an toàn cộng đồng cũng như khả năng truy xuất. Khi số lượng ảnh tăng, những thao tác tưởng như độc lập như tải ảnh, tạo ảnh thu nhỏ, làm mờ, đóng dấu bản quyền, kiểm duyệt, lập chỉ mục tìm kiếm và ghi nhận tương tác không thể tiếp tục đặt trực tiếp trong luồng yêu cầu đồng bộ. Nếu toàn bộ các công việc này được thực thi ngay trong request HTTP khi người dùng đăng tải, thời gian phản hồi sẽ kéo dài, tài nguyên CPU và bộ nhớ bị chiếm giữ, đồng thời khả năng thất bại cục bộ của một bước xử lý có thể làm hỏng toàn bộ quy trình xuất bản.

Từ thực tế đó, nghiên cứu này xác định bốn bài toán kỹ thuật cốt lõi cần được giải quyết trong một nền tảng chia sẻ ảnh trực tuyến hiện đại.

Thứ nhất là bài toán kiểm duyệt nội dung nhạy cảm hoặc độc hại. Với các cộng đồng ảnh có tính mở, người dùng có thể đăng tải nội dung không phù hợp với chuẩn cộng đồng, đặc biệt là nội dung NSFW. Phương pháp kiểm duyệt hoàn toàn thủ công có ưu điểm về khả năng đánh giá ngữ cảnh, song không phù hợp khi khối lượng ảnh tăng nhanh. Mỗi ảnh tải lên cần được mở, quan sát và phân loại bởi người điều phối, dẫn tới độ trễ xuất bản cao, chi phí nhân sự lớn và áp lực tâm lý cho đội ngũ kiểm duyệt. Hơn nữa, nếu nền tảng cho phép đăng nhiều ảnh trong một tác phẩm, một ảnh vi phạm trong tập ảnh có thể làm cả tác phẩm rơi vào trạng thái cần xử lý, đòi hỏi hệ thống phải lưu được trạng thái lỗi ở cấp ảnh thay vì chỉ ở cấp bài đăng. Vì vậy, hệ thống cần có một tầng kiểm duyệt tự động ban đầu để phát hiện rủi ro, đưa ảnh về trạng thái cần xem xét và chỉ chuyển những trường hợp cần thiết

cho quản trị viên.

Thứ hai là bài toán phát hiện ảnh trùng lặp hoặc đạo nhái. Trong môi trường sáng tạo số, ảnh có thể bị tải lại dưới dạng đã đổi kích thước, nén lại, cắt nhẹ, đổi định dạng hoặc thêm nhiễu. Các kỹ thuật băm mật mã như SHA-256 chỉ phù hợp để kiểm tra hai tệp nhị phân hoàn toàn giống nhau; chúng không nhận ra hai ảnh gần như giống hệt về mặt thị giác nhưng khác nhau ở một vài byte. Điều này làm cho nền tảng dễ bị lạm dụng để tái đăng tác phẩm của người khác, làm loãng kết quả tìm kiếm và gây khó khăn cho việc bảo vệ quyền lợi tác giả. Một hệ thống chia sẻ ảnh cần sử dụng phương pháp băm nhận thức, trong đó ảnh được ánh xạ thành dấu vân tay thị giác có khả năng bảo toàn đặc trưng hình ảnh ở mức cảm nhận, sau đó so sánh bằng khoảng cách Hamming để phát hiện trùng lặp gần.

Thứ ba là hạn chế của tìm kiếm ảnh dựa trên từ khóa. Tìm kiếm từ khóa truyền thống, còn gọi là lexical search, chủ yếu so khớp chuỗi trong tiêu đề, mô tả, tên tác giả và thẻ. Cách tiếp cận này hoạt động tốt khi người đăng mô tả chính xác nội dung ảnh và người tìm kiếm dùng cùng hệ từ vựng. Tuy nhiên, ảnh nghệ thuật thường chứa các yếu tố thị giác khó diễn đạt bằng từ khóa ngắn, ví dụ phong cách nét vẽ, bố cục, cảm xúc, ánh sáng, dáng nhân vật, hoặc tương đồng về hình khối giữa ảnh phác thảo và ảnh hoàn thiện. Khi người dùng nhập một truy vấn tự nhiên như “nhân vật fantasy với ánh sáng xanh và bố cục chân dung”, hệ thống tìm kiếm thuần văn bản khó hiểu được mối quan hệ ngữ nghĩa giữa câu truy vấn và nội dung ảnh. Do đó, nền tảng cần kết hợp tìm kiếm toàn văn với tìm kiếm ngữ nghĩa trong không gian vector, nơi ảnh và truy vấn được biểu diễn bằng embedding và được so sánh bằng độ tương đồng.

Thứ tư là thiếu cơ chế hỗ trợ người sáng tạo kiếm thêm thu nhập trực tiếp từ cộng đồng người theo dõi. Nhiều nền tảng chia sẻ ảnh chỉ dừng lại ở chức năng đăng tải miễn phí và tương tác xã hội. Mô hình này tạo ra giá trị cộng đồng nhưng chưa hình thành dòng doanh thu bền vững cho nghệ sĩ. Trong khi đó, người sáng tạo thường có nhu cầu cung cấp nội dung độc quyền, ảnh độ phân giải cao, bản phác thảo, tác phẩm hậu trường hoặc quyền truy cập sớm cho nhóm người hâm mộ trả phí. Nếu thiếu cơ chế membership hoặc tier, tác giả phải chuyển người dùng sang dịch vụ bên ngoài, làm đứt gãy trải nghiệm và khiến nền tảng không kiểm soát được quyền truy cập nội dung. Vì vậy, một hệ thống hoàn chỉnh cần tích hợp gói đăng ký, phân quyền nội dung theo tier và cổng thanh toán để kết nối trực tiếp giữa người sáng tạo và người ủng hộ.

Từ bốn bài toán trên, có thể thấy rằng việc xây dựng nền tảng chia sẻ ảnh trực tuyến không chỉ là bài toán giao diện hiển thị ảnh, mà là bài toán hệ thống bao gồm

xử lý dữ liệu đa phương tiện, kiểm duyệt tự động, tìm kiếm lai, quản lý phân quyền, xử lý thanh toán và vận hành bất đồng bộ. Đề tài này hướng tới xây dựng một nền tảng có khả năng kết hợp các thành phần nói trên trong một kiến trúc module thống nhất. Hệ thống không thay thế hoàn toàn vai trò của quản trị viên, nhưng sử dụng tự động hóa để giảm tải các tác vụ lặp lại, phát hiện sớm rủi ro và tạo cơ sở dữ liệu giàu ngữ nghĩa hơn cho các chức năng tìm kiếm, khuyến nghị và điều phối nội dung.

1.2 Mục tiêu và phạm vi đề tài

Mục tiêu tổng quát của đề án là xây dựng mã nguồn và thiết kế kỹ thuật cho một nền tảng web chia sẻ ảnh trực tuyến phục vụ cộng đồng nghệ thuật số. Hệ thống được triển khai theo mô hình ứng dụng web hiện đại, trong đó frontend sử dụng Next.js với App Router, React và hệ sinh thái Mantine để xây dựng giao diện, còn backend sử dụng NestJS, Prisma ORM và PostgreSQL để tổ chức nghiệp vụ, dữ liệu và các API phục vụ giao diện. Ngoài các chức năng xã hội cơ bản, hệ thống tập trung vào các năng lực tự động hóa gồm kiểm duyệt NSFW, phát hiện trùng lặp bằng pHash, tìm kiếm toàn văn bằng Meilisearch, tìm kiếm ngữ nghĩa bằng embedding vector và xử lý thanh toán qua Stripe.

Ở phía người dùng cuối, hệ thống cho phép duyệt danh sách tác phẩm, xem chi tiết tác phẩm, tìm kiếm theo từ khóa, tìm kiếm bằng ngôn ngữ tự nhiên, tương tác thông qua lượt thích, bình luận, đánh dấu và bộ sưu tập. Các chức năng này được thiết kế để phục vụ hành vi sử dụng phổ biến của cộng đồng chia sẻ ảnh: người dùng có thể khám phá nội dung mới, theo dõi nghệ sĩ, lưu lại tác phẩm quan tâm và tương tác với tác giả. Với người dùng đã đăng nhập, hệ thống gắn định danh ứng dụng với định danh Clerk, từ đó đồng bộ hồ sơ người dùng vào cơ sở dữ liệu nội bộ để phục vụ phân quyền và ghi nhận tương tác.

Ở phía người sáng tạo, hệ thống hỗ trợ đăng tải tác phẩm gồm một hoặc nhiều ảnh, nhập tiêu đề, mô tả, thẻ, đánh dấu tác phẩm AI, bật watermark và quản lý trạng thái xuất bản. Sau khi người sáng tạo tải ảnh lên, hệ thống không xuất bản ngay lập tức mà đưa tác phẩm vào trạng thái xử lý nền. Worker sẽ tải tệp gốc từ storage, kiểm tra trùng lặp, kiểm duyệt NSFW, tạo các biến thể ảnh như bản xem trước, ảnh thu nhỏ và ảnh làm mờ, sau đó ghi kết quả vào cơ sở dữ liệu. Nếu tất cả ảnh hợp lệ, tác phẩm được chuyển sang trạng thái `PUBLISHED`; nếu tồn tại ảnh vi phạm hoặc trùng lặp, tác phẩm được đưa vào trạng thái `ACTION_REQUIRED` để người dùng và quản trị viên có thể xử lý tiếp.

Ở phía kinh tế sáng tạo, hệ thống định nghĩa người sáng tạo như một vai trò mở rộng của người dùng thường. Một người dùng có thể trở thành artist, tạo các gói

ArtistTier, cấu hình giá, mô tả quyền lợi và giới hạn thành viên. Tác phẩm có thể được đặt ở chế độ công khai hoặc TIER_GATED; khi đó quyền truy cập nội dung phụ thuộc vào trạng thái đăng ký tier của người xem. Mô hình này cho phép nền tảng hỗ trợ quan hệ trực tiếp giữa nghệ sĩ và người hâm mộ mà không cần chuyển quyền truy cập sang hệ thống ngoài. Thanh toán và quản lý vòng đời đăng ký được tích hợp với Stripe, còn trạng thái đăng ký được đồng bộ về cơ sở dữ liệu để backend kiểm tra quyền truy cập.

Ở phía quản trị, hệ thống phục vụ các vai trò điều phối như Moderator, Admin và Super Admin. Các vai trò này có thể xem báo cáo vi phạm, xử lý tác phẩm cần kiểm duyệt, quản lý người dùng bị cảnh báo hoặc bị khóa, theo dõi audit log, cấu hình ngưỡng kiểm duyệt, quản lý thông báo và giám sát một số chỉ số vận hành. Backend sử dụng guard xác thực Clerk và guard phân quyền theo role để bảo vệ các API nhạy cảm. Điều này tách rõ người dùng phổ thông khỏi nhóm vận hành, đồng thời vẫn giữ được lịch sử thao tác phục vụ truy vết.

Phạm vi của đề án được giới hạn ở việc xây dựng nền tảng web và các thành phần server-side cốt lõi. Đề tài không đặt mục tiêu huấn luyện lại mô hình thị giác từ đầu; thay vào đó, hệ thống tích hợp các mô hình và thư viện có sẵn như NSFWJS, Transformers.js/CLIP hoặc dịch vụ embedding tương thích để phục vụ xử lý ảnh. Đề tài cũng không đi sâu vào tối ưu hạ tầng phân tán quy mô rất lớn như sharding cơ sở dữ liệu, multi-region deployment hay CDN chuyên dụng toàn cầu. Những nội dung này được xem là hướng mở rộng sau khi kiến trúc ứng dụng, mô hình dữ liệu và luồng nghiệp vụ chính đã ổn định.

Từ góc độ tác nhân, phạm vi hệ thống gồm bốn nhóm chính. Khách là người chưa đăng nhập, có thể truy cập nội dung công khai, tìm kiếm và xem chi tiết một số tác phẩm theo chính sách công khai. Người dùng thường là người đã đăng nhập, có thể tương tác với tác phẩm, quản lý hồ sơ cá nhân, bình luận, thích, lưu tác phẩm và đăng ký gói thành viên. Người sáng tạo là người dùng có thêm khả năng đăng tải và quản lý tác phẩm, thiết lập tier, quản lý nội dung dành riêng cho người ủng hộ và theo dõi doanh thu liên quan. Quản trị viên hoặc điều phối viên là nhóm chịu trách nhiệm kiểm duyệt, quản lý báo cáo, cấu hình hệ thống và đảm bảo nền tảng vận hành theo quy định cộng đồng.

1.3 Định hướng giải pháp

Định hướng giải pháp của đề tài là xây dựng hệ thống theo kiến trúc modular monolith. Trong kiến trúc này, ứng dụng backend được triển khai như một tiến trình hoặc một đơn vị triển khai chính, nhưng mã nguồn được chia thành các module nghiệp vụ độc lập tương đối rõ ràng. Codebase hiện tại thể hiện

định hướng này thông qua các module NestJS như AuthModule, UsersModule, ArtworksModule, StorageModule, QueueModule, SearchModule, StatsModule, ReportsModule, AdminModule, PaymentsModule, DuplicateDetectionModule, AiSearchModule và RecommendationModule. Cách tổ chức này phù hợp với đồ án ứng dụng vì giảm độ phức tạp triển khai so với microservices, đồng thời vẫn giữ được ranh giới nghiệp vụ để có thể tách dịch vụ trong tương lai nếu tải hệ thống tăng.

Lớp backend sử dụng NestJS làm framework chính. NestJS hỗ trợ mô hình module, dependency injection, provider, controller và guard, nhờ đó các thành phần như Prisma service, Redis client, Clerk strategy, Stripe service, Search service và các worker xử lý nền có thể được tiêm phụ thuộc một cách nhất quán. Prisma ORM được sử dụng để ánh xạ mô hình dữ liệu TypeScript với PostgreSQL. Trong lược đồ dữ liệu, các thực thể quan trọng gồm User, Artwork, ArtworkImage, Tag, Like, Comment, Collection, Report, ArtistTier, TierSubscription, Payment và Payout. Đặc biệt, bảng artwork_images có các trường phash, embedding vector (512) và has_embedding, cho thấy hệ thống lưu đồng thời dấu vân tay nhận thức và biểu diễn vector của ảnh.

Đối với các tác vụ tiêu tốn tài nguyên, hệ thống sử dụng mô hình background worker thông qua BullMQ và Redis. Khi người dùng đăng tải tác phẩm, API backend chỉ tạo bản ghi tác phẩm ban đầu, lưu file tạm và đưa job vào hàng đợi. Worker ArtworkProcessor đảm nhiệm chuỗi xử lý ảnh phía sau. Cách tiếp cận này tránh việc request HTTP bị treo trong thời gian dài, giảm nguy cơ timeout và cho phép hệ thống báo cáo tiến độ theo từng pha như tải ảnh, kiểm tra trùng lặp, kiểm duyệt NSFW, xử lý ảnh, lưu cơ sở dữ liệu, lập chỉ mục tìm kiếm và hoàn tất. Redis đóng vai trò backend cho hàng đợi, đồng thời được dùng cho cache embedding truy vấn, chống spam lượt xem và buffer cập nhật chỉ số sang Meilisearch.

Đối với tìm kiếm, hệ thống kết hợp hai hướng bổ sung cho nhau. Meilisearch được sử dụng cho tìm kiếm toàn văn, hỗ trợ typo tolerance, ranking rule, filter theo tags, rating, tác giả, độ phân giải và sắp xếp theo thời gian, lượt thích hoặc lượt xem. Đây là lựa chọn phù hợp cho truy vấn có từ khóa rõ ràng và cần phản hồi nhanh. Bên cạnh đó, PostgreSQL được mở rộng bằng pgvector để lưu embedding của ảnh trong không gian 512 chiều. Với truy vấn ngôn ngữ tự nhiên hoặc ảnh phác thảo, hệ thống sinh embedding tương ứng, sau đó dùng toán tử khoảng cách cosine của pgvector để tìm các ảnh gần nhất. Hai cơ chế này giúp hệ thống vừa phục vụ tìm kiếm từ khóa truyền thống, vừa có khả năng hiểu tương đồng ngữ nghĩa và thị giác.

Đối với kiểm duyệt và phát hiện trùng lặp, hệ thống triển khai kiểm tra trong pipeline xử lý ảnh. Ảnh đầu vào được tính pHash bằng thư viện `sharp-phash`; kết quả được so sánh với các pHash đã lưu trong cơ sở dữ liệu bằng khoảng cách Hamming. Nếu khoảng cách nhỏ hơn ngưỡng, ảnh được đánh dấu trùng lặp và không được xuất bản tự động. Sau đó, ảnh được resize về kích thước phù hợp, chuyển thành tensor và phân loại bằng NSFWS chạy trên TensorFlow.js backend CPU. Nếu xác suất thuộc nhóm nhạy cảm vượt ngưỡng cấu hình trong `System-Setting`, ảnh được lưu với metadata lỗi, ảnh làm mờ được tạo để phục vụ xem xét và tác phẩm chuyển sang trạng thái cần xử lý. Như vậy, hệ thống không chỉ từ chối thô bạo ở tầng API, mà lưu lại đủ thông tin để hỗ trợ quy trình kiểm duyệt hậu kỳ.

Đối với thanh toán, hệ thống tích hợp Stripe để tạo checkout session, customer portal, quản lý subscription nền tảng và subscription theo artist tier. Backend lưu `stripeCustomerId` trong bảng người dùng, đồng bộ trạng thái đăng ký sau webhook và ánh xạ các trạng thái Stripe như `active`, `trialing`, `past_due`, `canceled` sang trạng thái nội bộ như `ACTIVE`, `PAST_DUE`, `CANCELED` và `EXPIRED`. Thiết kế này làm cho quyền truy cập nội dung trả phí được quyết định bởi dữ liệu nội bộ ổn định, trong khi Stripe đảm nhiệm phần thanh toán, hóa đơn và cổng quản lý phương thức thanh toán.

Tổng thể, định hướng giải pháp của đề án là kết hợp một kiến trúc ứng dụng module, một pipeline xử lý nền bất đồng bộ và một lớp dữ liệu đa dạng gồm dữ liệu quan hệ, chỉ mục toàn văn, cache và vector embedding. Sự kết hợp này phù hợp với bản chất của nền tảng chia sẻ ảnh: dữ liệu có tính xã hội, nội dung có tính đa phương tiện, truy vấn có tính ngữ nghĩa và vận hành cần tự động hóa để giảm tải cho con người.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp được tổ chức thành năm chương tiếp theo.

Chương 2 trình bày khảo sát và phân tích yêu cầu của hệ thống. Chương này dự kiến khảo sát các nền tảng chia sẻ ảnh và cộng đồng nghệ thuật số hiện có, phân tích các nhóm tác nhân, mô tả chức năng chính, yêu cầu phi chức năng và các ràng buộc nghiệp vụ liên quan tới kiểm duyệt, tìm kiếm, đăng tải và thanh toán.

Chương 3 trình bày nền tảng lý thuyết và công nghệ sử dụng. Nội dung chương tập trung vào kiến trúc backend NestJS, Prisma và PostgreSQL; kiến trúc frontend Next.js App Router, Mantine và Zustand; các nền tảng học máy và thuật toán gồm CLIP, NSFWS, pHash, không gian vector và `pgvector`; cuối cùng là các dịch

vụ trung gian như BullMQ, Redis, Meilisearch, Clerk và Stripe.

Chương 4 trình bày phân tích thiết kế, triển khai và đánh giá hệ thống. Chương này dự kiến mô tả kiến trúc tổng thể, thiết kế cơ sở dữ liệu, thiết kế API, thiết kế giao diện, pipeline xử lý ảnh, quy trình triển khai và phương pháp đánh giá hệ thống.

Chương 5 trình bày các giải pháp và đóng góp nổi bật của đề án. Nội dung chương dự kiến đi sâu vào các điểm kỹ thuật có giá trị như tách xử lý ảnh khỏi luồng chính, kiểm duyệt bán tự động, phát hiện trùng lặp bằng pHash, tìm kiếm ngữ nghĩa bằng vector embedding, đồng bộ chỉ mục tìm kiếm và mô hình thành viên cho người sáng tạo.

Chương 6 tổng kết kết quả đạt được, nêu các hạn chế còn tồn tại và đề xuất hướng phát triển trong tương lai. Các hướng phát triển có thể bao gồm tối ưu chỉ mục vector, mở rộng bộ lọc kiểm duyệt, cải tiến khuyến nghị cá nhân hóa, bổ sung hệ thống giải quyết tranh chấp bản quyền và triển khai hạ tầng phân tán cho quy mô lớn hơn.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

2.1 Khảo sát hiện trạng

- 2.1.1 Các nền tảng chia sẻ ảnh và cộng đồng nghệ thuật số**
- 2.1.2 Quy trình kiểm duyệt nội dung trên các nền tảng hiện có**
- 2.1.3 Cơ chế tìm kiếm và khám phá nội dung**
- 2.1.4 Mô hình kiếm tiền của người sáng tạo**
- 2.1.5 Tổng kết hạn chế**

2.2 Tổng quan tác nhân và chức năng

- 2.2.1 Tác nhân Khách**
- 2.2.2 Tác nhân Người dùng thường**
- 2.2.3 Tác nhân Người sáng tạo**
- 2.2.4 Tác nhân Quản trị viên và Điều phối viên**
- 2.2.5 Biểu đồ use case tổng quan**

2.3 Đặc tả yêu cầu chức năng

- 2.3.1 Đăng ký, đăng nhập và đồng bộ tài khoản**
- 2.3.2 Đăng tải và quản lý tác phẩm**
- 2.3.3 Kiểm duyệt và xử lý báo cáo**
- 2.3.4 Tìm kiếm, khám phá và tương tác**
- 2.3.5 Quản lý gói thành viên và thanh toán**

2.4 Yêu cầu phi chức năng

- 2.4.1 Hiệu năng và khả năng đáp ứng**
- 2.4.2 An toàn, bảo mật và phân quyền**
- 2.4.3 Độ tin cậy của xử lý nền**
- 2.4.4 Khả năng mở rộng và bảo trì**
- 2.4.5 Khả năng quan sát và truy vết**

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

3.1 Nền tảng Backend và cơ sở dữ liệu

3.1.1 Kiến trúc NestJS Framework

NestJS là framework backend cho Node.js được thiết kế xoay quanh TypeScript, decorator, dependency injection và tư duy module hóa [1]. Trong hệ thống đề xuất, NestJS không chỉ đóng vai trò lớp API nhận request từ frontend, mà còn là bộ khung tổ chức toàn bộ nghiệp vụ server-side: xác thực, phân quyền, quản lý người dùng, quản lý tác phẩm, xử lý ảnh, tìm kiếm, thanh toán, thống kê và quản trị. Điểm phù hợp của NestJS đối với đề án nằm ở chỗ framework này tạo ra ranh giới rõ giữa controller, service, provider, module và guard, giúp hệ thống có thể phát triển theo modular monolith thay vì trộn toàn bộ logic nghiệp vụ trong một lớp ứng dụng duy nhất.

Khái niệm trung tâm đầu tiên của NestJS là *module*. Một module gom nhóm các controller và provider có liên quan về mặt nghiệp vụ. Trong codebase, AppModule là module gốc và nhập các module con như AuthModule, UsersModule, ArtworksModule, QueueModule, SearchModule, StatsModule, ReportsModule, AdminModule, PaymentsModule, DuplicateDetectionModule và AiSearchModule. Cách chia này phản ánh trực tiếp cấu trúc domain của nền tảng chia sẻ ảnh. Ví dụ, ArtworksModule chịu trách nhiệm các nghiệp vụ xoay quanh tác phẩm; QueueModule chịu trách nhiệm hàng đợi xử lý ảnh; SearchModule chịu trách nhiệm lập chỉ mục và tìm kiếm toàn văn; AiSearchModule chịu trách nhiệm sinh và truy vấn vector embedding. Như vậy, mặc dù ứng dụng vẫn được triển khai như một monolith, mỗi miền nghiệp vụ vẫn có ranh giới mã nguồn và phụ thuộc rõ ràng.

Khái niệm trung tâm thứ hai là *provider*. Provider trong NestJS thường là service hoặc factory có thể được container dependency injection khởi tạo và tiêm vào các thành phần khác. Ví dụ, PrismaService được khai báo trong PrismaModule và có thể được sử dụng bởi nhiều service nghiệp vụ để truy cập cơ sở dữ liệu. Tương tự, Redis client được tạo trong RedisModule thông qua một injection token REDIS_CLIENT, sau đó được tiêm vào các service cần cache hoặc buffer. Cơ chế dependency injection này giúp tách phần khởi tạo tài nguyên khỏi phần sử dụng tài nguyên. Khi một service như AiSearchService cần cả PrismaService, EmbeddingService và Redis client, các phụ thuộc được khai báo rõ trong constructor thay vì tự khởi tạo thủ công, làm giảm coupling và tăng khả năng kiểm thử.

Khái niệm thứ ba là *controller*. Controller định nghĩa các endpoint HTTP và ánh xạ request tới service tương ứng. Trong một nền tảng chia sẻ ảnh, controller cần xử lý các nhóm API như đăng tải tác phẩm, lấy danh sách tác phẩm, tìm kiếm, tạo report, quản lý thanh toán hoặc thao tác admin. NestJS tách controller khỏi service để controller chỉ đảm nhiệm xử lý request/response, validate tham số, gắn guard, còn service thực thi nghiệp vụ. Sự phân tách này đặc biệt quan trọng trong các luồng phức tạp như đăng tải ảnh: controller nhận file metadata và yêu cầu tạo tác phẩm, nhưng các bước xử lý nặng như kiểm duyệt, sinh thumbnail và lập chỉ mục được đẩy sang worker thông qua service hàng đợi.

Khái niệm thứ tư là *guard*. Guard quyết định một request có được phép đi tiếp hay không. Hệ thống sử dụng Clerk để xác thực người dùng, do đó backend định nghĩa ClerkStrategy để xác minh JWT và ClerkGuard để bảo vệ route. Khi token hợp lệ, strategy lấy định danh clerkId, gọi Clerk API để lấy hồ sơ người dùng và thực hiện lazy sync vào bảng users. Cách tiếp cận này giải quyết được hai yêu cầu đồng thời: hệ thống không phải tự xây dựng toàn bộ cơ chế đăng nhập, nhưng vẫn có bản ghi người dùng nội bộ để lưu vai trò, trạng thái cấm, stripeCustomerId, tương tác và quan hệ subscription. Đối với các route quản trị, hệ thống có thể kết hợp ClerkGuard với guard phân quyền theo vai trò như MODERATOR, ADMIN hoặc SUPER_ADMIN.

Ở mức kiến trúc, NestJS cho phép hệ thống áp dụng nguyên tắc đảo ngược phụ thuộc. Các module cấp cao như ArtworksModule không cần biết chi tiết cách Redis kết nối, cách Prisma mở connection pool hay cách Stripe khởi tạo client; chúng chỉ phụ thuộc vào interface hành vi do provider cung cấp. Với đồ án này, đặc điểm đó rất quan trọng vì các thành phần bên ngoài khá đa dạng: PostgreSQL, Redis, Meilisearch, Clerk, Stripe, storage và worker AI. Nếu các phụ thuộc này bị gọi trực tiếp rải rác trong controller, hệ thống sẽ khó bảo trì, khó thay thế và khó kiểm thử. Ngược lại, khi được tổ chức thành provider, việc cấu hình môi trường, thay đổi implementation hoặc mock trong unit test trở nên có kiểm soát hơn.

3.1.2 Prisma ORM

Prisma là ORM hiện đại cho TypeScript, cung cấp lược đồ khai báo, client sinh mã tự động và cơ chế migration cho cơ sở dữ liệu quan hệ [2]. Trong hệ thống đề xuất, Prisma đóng vai trò cầu nối giữa mô hình dữ liệu PostgreSQL và mã nguồn NestJS. File schema.prisma định nghĩa các model như User, Artwork, ArtworkImage, Tag, Comment, Like, Collection, Report, ArtistTier, TierSubscription, Subscription, Payment và Payout. Mỗi model không chỉ định nghĩa trường dữ liệu mà còn mô tả quan hệ, chỉ

mục, khóa duy nhất và ánh xạ tên cột sang cơ sở dữ liệu vật lý.

Ưu điểm chính của Prisma trong hệ thống là tính an toàn kiểu. Khi schema khai báo `User.role` thuộc enum `UserRole`, TypeScript client sinh ra sẽ giúp service chỉ truyền các giá trị hợp lệ như `USER`, `MODERATOR`, `ADMIN` hoặc `SUPER_ADMIN`. Khi `Artwork.status` thuộc enum `ArtworkStatus`, các trạng thái như `PROCESSING`, `PUBLISHED`, `FAILED`, `ACTION_REQUIRED` được kiểm soát ở tầng mã nguồn. Điều này làm giảm lỗi do truyền chuỗi tự do, đặc biệt trong các workflow có nhiều trạng thái như xử lý ảnh.

Prisma cũng hỗ trợ truy vấn quan hệ rõ ràng thông qua `include` và `select`. Ví dụ, khi lập chỉ mục một tác phẩm vào Meilisearch, service cần lấy thông tin tác phẩm, tác giả và tag. Với Prisma, truy vấn có thể biểu diễn trực tiếp dưới dạng lấy `artwork` kèm `author` và `tags.include.tag`. Cách viết này giúp mã nguồn gần với mô hình domain hơn so với việc tự nối bảng bằng SQL trong mọi trường hợp.

Tuy nhiên, hệ thống cũng cho thấy một giới hạn thực tế của ORM: không phải mọi kiểu dữ liệu đặc thù của PostgreSQL đều được Prisma hỗ trợ như kiểu gốc. Cột `embedding` trong bảng `artwork_images` được khai báo là `Unsupported("vector(512)")`. Điều này có nghĩa Prisma vẫn quản lý được model tổng thể, nhưng các thao tác đọc/ghi vector cần dùng raw SQL. Trong `aiSearchService`, hệ thống lưu embedding bằng câu lệnh:

```
1 UPDATE artwork_images
2 SET embedding = $1::vector, has_embedding = true
3 WHERE id = $2
```

Listing 3.1: Cập nhật vector embedding trong PostgreSQL

Tương tự, truy vấn tìm kiếm vector sử dụng toán tử khoảng cách của `pgvector` nên cũng được thực hiện bằng `$queryRawUnsafe`. Đây là một lựa chọn thực dụng: dùng Prisma cho phần lớn dữ liệu quan hệ để tận dụng type safety, nhưng dùng SQL trực tiếp cho các thao tác cần khả năng đặc thù của cơ sở dữ liệu.

3.1.3 PostgreSQL và mô hình dữ liệu quan hệ

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở có độ ổn định cao, hỗ trợ transaction, chỉ mục đa dạng, kiểu dữ liệu mở rộng và cơ chế extension [3]. Trong hệ thống chia sẻ ảnh, PostgreSQL được chọn làm nguồn dữ liệu chính vì dữ liệu nghiệp vụ có quan hệ chặt chẽ và yêu cầu tính nhất quán cao. Một tác phẩm thuộc về một tác giả, có nhiều ảnh, nhiều tag, nhiều bình luận, nhiều lượt thích, nhiều bookmark và có thể liên quan đến report hoặc tier nội dung. Các quan

hệ này phù hợp với mô hình quan hệ hơn so với lưu trữ tài liệu phi cấu trúc thuần túy.

Model User là trung tâm của hệ thống định danh. Bản ghi người dùng lưu `clerkId` để liên kết với Clerk, email, username, thông tin hồ sơ, vai trò, trạng thái artist, trạng thái cấm và `stripeCustomerId`. Từ User, hệ thống mở rộng sang các quan hệ xã hội như Follow, Like, Comment, Collection, đồng thời mở rộng sang các quan hệ kinh tế như Subscription, ArtistTier, Tier-Subscription, Payment và Payout. Thiết kế này cho phép một người dùng vừa là người xem, vừa có thể trở thành artist, vừa có thể là subscriber của artist khác.

Model Artwork biểu diễn đơn vị nội dung chính. Một tác phẩm có tiêu đề, mô tả, loại, trạng thái, rating, thông tin AI, bộ đếm lượt xem/lượt thích/lượt bình luận, lớp tỷ lệ ảnh, độ phân giải lớn nhất, cờ độ phân giải cao, visibility và tier yêu cầu. Trạng thái PROCESSING hoặc ACTION_REQUIRED cho phép hệ thống biểu diễn các giai đoạn trung gian thay vì chỉ có hai trạng thái đăng hoặc không đăng. Điều này phù hợp với pipeline xử lý ảnh bất đồng bộ, nơi tác phẩm có thể được tạo trước khi ảnh hoàn tất kiểm duyệt.

Model ArtworkImage lưu dữ liệu ở cấp ảnh. Đây là điểm quan trọng vì một tác phẩm có thể chứa nhiều ảnh và mỗi ảnh có thể có kết quả xử lý khác nhau. Các trường `originalUrl`, `url`, `thumbnailUrl`, `blurredUrl`, `width`, `height`, `aspectRatio`, `status` và `errorMetadata` cho phép hệ thống lưu đầy đủ các biến thể ảnh và lỗi kiểm duyệt. Trường `hash` phục vụ phát hiện trùng lặp, còn `embedding` phục vụ tìm kiếm ngữ nghĩa. Việc đặt `hash` và `embedding` ở cấp ảnh thay vì cấp tác phẩm là hợp lý vì một tác phẩm nhiều ảnh có thể chứa các ảnh với nội dung thị giác hoàn toàn khác nhau.

PostgreSQL cũng hỗ trợ transaction, và hệ thống tận dụng điều này trong bước lưu kết quả xử lý ảnh. Worker tạo nhiều bản ghi ArtworkImage và cập nhật trạng thái Artwork trong cùng một transaction. Nếu một phần ghi dữ liệu thất bại, transaction có thể rollback để tránh trạng thái nửa vời, ví dụ tác phẩm đã chuyển sang PUBLISHED nhưng ảnh chưa được ghi đủ. Trong các hệ thống đa phương tiện, tính nguyên tử ở bước cập nhật metadata là cần thiết vì file storage, search index và database không luôn có transaction phân tán với nhau.

3.2 Nền tảng Frontend và State Management

3.2.1 Next.js App Router

Next.js là framework React hỗ trợ rendering phía server, routing theo file system, tối ưu bundle, server actions, API route và nhiều cơ chế tối ưu trải nghiệm web

[4]. Hệ thống frontend sử dụng Next.js App Router, thể hiện qua thư mục `frontend/src/app` với các route như `discover`, `search`, `upload`, `dashboard`, `admin`, `artworks/[id]`, `artists/[username]` và `checkout`. App Router tổ chức giao diện theo cây layout lồng nhau, cho phép mỗi vùng route có layout, loading, error boundary và metadata riêng.

Một điểm cốt lõi của App Router là phân biệt Server Components và Client Components. Server Components được render trên server, có thể truy cập dữ liệu server-side và không gửi toàn bộ logic component xuống trình duyệt. Điều này giúp giảm kích thước JavaScript phía client và phù hợp với các trang chủ yếu hiển thị dữ liệu như trang chi tiết tác phẩm hoặc trang hồ sơ artist. Ngược lại, Client Components được đánh dấu bằng chỉ thị `'use client'` và cần thiết khi component sử dụng state phía trình duyệt, effect, event handler, context hoặc thư viện phụ thuộc vào DOM. Trong codebase, các provider như `MantineProvider` và `QueryProvider` là Client Components vì chúng thiết lập context React, quản lý cache phía client và xử lý tương tác giao diện.

Root layout của frontend đặt ngôn ngữ mặc định, thiết lập script màu sắc Mantine trước khi render và bao bọc nội dung bằng các provider ở layout theo locale. `Layout[locale]/layout.tsx` sử dụng `ClerkProvider`, `MantineProvider`, `QueryProvider` và `NextIntlClientProvider`. Cách tổ chức này cho phép mọi route con cùng chia sẻ xác thực, theme, cache truy vấn và quốc tế hóa. Đồng thời, việc đặt route theo `[locale]` giúp hệ thống có thể mở rộng giao diện đa ngôn ngữ mà không phải nhân bản toàn bộ cấu trúc trang.

Middleware của frontend sử dụng Clerk middleware kết hợp `next-intl`. Các route như `/dashboard` và `/admin` được bảo vệ bằng `createRouteMatcher`. Khi người dùng đã đăng nhập, middleware gọi API backend `/users/me` với Bearer token để kiểm tra trạng thái cấm; nếu người dùng bị ban, request được chuyển hướng sang trang `/ban`. Cách kiểm tra này thể hiện sự phối hợp giữa xác thực bên ngoài và trạng thái nghiệp vụ nội bộ: Clerk xác nhận danh tính, còn backend quyết định chính sách nền tảng như cấm tài khoản.

3.2.2 Mantine UI

Mantine là thư viện UI React cung cấp component, hook, theme và hệ thống style nhất quán. Trong hệ thống, Mantine được dùng để xây dựng giao diện có tính ứng dụng cao: bảng quản trị, form upload, điều khiển lọc, progress bar, modal, notification, layout và các thành phần tương tác. File `MantineProvider.tsx` định nghĩa theme riêng cho hệ thống, trong đó màu chính là dải `rose/pink`, font chữ là Plus Jakarta Sans, các mức radius được chuẩn hóa và `respectReducedMo-`

tion được bật để tôn trọng thiết lập giảm chuyển động của người dùng.

Về mặt kỹ thuật, Mantine giúp giảm chi phí xây dựng lại các primitive giao diện thường gặp như input, select, table, tabs, modal và notification. Điều này đặc biệt có ý nghĩa trong các trang nghiệp vụ như admin moderation, dashboard membership hoặc upload artwork, nơi độ chính xác và khả năng thao tác quan trọng hơn trang trí. Khi hệ thống có nhiều vai trò người dùng, việc duy trì một hệ UI thống nhất cũng làm giảm sai khác trải nghiệm giữa khu vực công khai, dashboard artist và trang quản trị.

3.2.3 Zustand và phân loại trạng thái frontend

Trong ứng dụng web hiện đại, không phải mọi trạng thái đều có cùng bản chất. Có thể phân biệt tối thiểu ba nhóm: trạng thái server, trạng thái URL và trạng thái UI cục bộ. Trạng thái server là dữ liệu lấy từ backend như danh sách tác phẩm, thông tin user, subscription hoặc kết quả tìm kiếm; nhóm này phù hợp với TanStack Query vì cần cache, stale time, retry và invalidation. Trạng thái URL là tham số truy vấn như từ khóa, bộ lọc, trang hoặc sort; nhóm này nên nằm trong URL để có thể chia sẻ và reload. Trạng thái UI cục bộ là những thông tin như sidebar đang mở hay đóng, modal nào đang bật, hoặc trạng thái loading nội bộ; nhóm này phù hợp với store nhẹ như Zustand.

Zustand là thư viện state management tối giản cho React. Trong codebase, useUIStore định nghĩa các trạng thái sidebarOpen, activeModal và isLoading, đồng thời sử dụng middleware persist để lưu một phần state vào local storage. So với các mô hình state management nặng, Zustand phù hợp với nhu cầu của hệ thống vì nó không áp đặt boilerplate phức tạp, không yêu cầu reducer toàn cục và dễ chia nhỏ store theo miền giao diện. Trong ngữ cảnh đồ án, Zustand được dùng cho trạng thái UI, còn dữ liệu nghiệp vụ vẫn được lấy và đồng bộ qua API để tránh tạo hai nguồn sự thật cho cùng một loại dữ liệu.

3.3 Lý thuyết về trí tuệ nhân tạo và thuật toán

3.3.1 Mô hình CLIP và học biểu diễn đa phương thức

CLIP, viết tắt của Contrastive Language-Image Pretraining, là một mô hình học biểu diễn đa phương thức do OpenAI đề xuất, có khả năng đưa ảnh và văn bản vào cùng một không gian vector [5]. Ý tưởng cốt lõi của CLIP là thay vì huấn luyện một mô hình phân loại ảnh với tập nhãn cố định, mô hình được huấn luyện trên các cặp ảnh-văn bản ở quy mô lớn. Mỗi ảnh đi qua một Image Encoder, mỗi mô tả văn bản đi qua một Text Encoder. Hai encoder này tạo ra hai vector embedding cùng chiều, sau đó được huấn luyện sao cho ảnh và câu mô tả đúng của nó nằm gần nhau, còn ảnh và các câu không tương ứng nằm xa nhau trong không gian embedding.

Về kiến trúc, Image Encoder có thể là mạng CNN hoặc Vision Transformer, còn Text Encoder thường là Transformer xử lý chuỗi token. Giả sử một batch huấn luyện gồm N cặp ảnh-văn bản $\{(I_i, T_i)\}_{i=1}^N$. Image Encoder f_θ ánh xạ ảnh I_i thành vector:

$$\mathbf{v}_i = \frac{f_\theta(I_i)}{\|f_\theta(I_i)\|_2}$$

Text Encoder g_ϕ ánh xạ văn bản T_i thành vector:

$$\mathbf{t}_i = \frac{g_\phi(T_i)}{\|g_\phi(T_i)\|_2}$$

Việc chuẩn hóa L2 đưa các vector về mặt cầu đơn vị, nhờ đó tích vô hướng giữa hai vector tương đương với cosine similarity. Độ tương đồng giữa ảnh i và văn bản j được tính bởi:

$$s_{ij} = \frac{\mathbf{v}_i^\top \mathbf{t}_j}{\tau}$$

Trong đó τ là tham số nhiệt độ, điều chỉnh độ sắc nét của phân phối xác suất. Với mỗi ảnh, văn bản đúng là T_i ; với mỗi văn bản, ảnh đúng là I_i . Hàm mất mát contrastive hai chiều có thể viết như sau:

$$\mathcal{L}_{I \rightarrow T} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(s_{ii})}{\sum_{j=1}^N \exp(s_{ij})} \quad (3.1)$$

$$\mathcal{L}_{T \rightarrow I} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(s_{ii})}{\sum_{j=1}^N \exp(s_{ji})} \quad (3.2)$$

$$\mathcal{L}_{CLIP} = \frac{1}{2} (\mathcal{L}_{I \rightarrow T} + \mathcal{L}_{T \rightarrow I}) \quad (3.3)$$

Cách huấn luyện này biến bài toán học biểu diễn thành bài toán phân biệt cặp đúng trong batch. Với mỗi ảnh, mô hình phải chọn đúng mô tả trong N mô tả; với mỗi mô tả, mô hình phải chọn đúng ảnh trong N ảnh. Khi N lớn, số lượng cặp âm trong mỗi batch tăng, buộc mô hình học các đặc trưng ngữ nghĩa bền vững hơn thay vì chỉ học nhãn cố định.

Trong hệ thống đề xuất, tinh thần của CLIP được sử dụng cho tìm kiếm ngữ nghĩa và tìm kiếm bằng phác thảo. Mỗi ảnh sau xử lý được sinh embedding 512

chiều và lưu trong cột `embedding` của bảng `artwork_images`. Khi người dùng nhập truy vấn văn bản hoặc cung cấp ảnh phác thảo, hệ thống sinh embedding truy vấn trong cùng không gian vector, sau đó so sánh với embedding ảnh đã lưu. Độ tương đồng cosine giữa hai vector $\mathbf{x}$ và $\mathbf{y}$ được định nghĩa:

$$\text{cosine}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2} = \frac{\sum_{k=1}^d x_k y_k}{\sqrt{\sum_{k=1}^d x_k^2} \sqrt{\sum_{k=1}^d y_k^2}} \quad (3.4)$$

Với $d = 512$, mỗi ảnh được biểu diễn bởi một điểm trong không gian 512 chiều. Hai ảnh hoặc một ảnh và một truy vấn văn bản càng gần nhau về ngữ nghĩa thì cosine similarity càng lớn. Trong codebase, truy vấn SQL dùng toán tử khoảng cách cosine `<=>` của `pgvector`, sau đó chuyển khoảng cách thành độ tương đồng bằng biểu thức $1 - \text{distance}$. Cách làm này phù hợp với mô hình CLIP vì embedding đã được thiết kế để so sánh trong một không gian chung giữa ảnh và ngôn ngữ.

3.3.2 Mô hình NSFWJS và phân loại hình ảnh

NSFWJS là thư viện phân loại ảnh nhạy cảm chạy trên TensorFlow.js. Về bản chất, bài toán NSFW là một bài toán phân loại hình ảnh đa lớp. Đầu vào là ảnh I , đầu ra là phân phối xác suất trên tập nhãn như `Porn`, `Hentai`, `Sexy`, `Neutral` và `Drawings`. Nếu gọi mô hình là h_ψ , kết quả dự đoán có thể viết:

$$\mathbf{p} = h_\psi(I), \quad \sum_{c \in \mathcal{C}} p_c = 1$$

Trong đó $\mathcal{C}$ là tập lớp và p_c là xác suất ảnh thuộc lớp c . Hệ thống tính điểm NSFW bằng cách lấy xác suất của các lớp nhạy cảm nhất, cụ thể là `Porn` hoặc `Hentai`, rồi so sánh với ngưỡng cấu hình. Nếu:

$$\max(p_{\text{Porn}}, p_{\text{Hentai}}) > \theta_{\text{NSFW}}$$

thì ảnh bị đánh dấu vi phạm và tác phẩm chuyển sang trạng thái cần xử lý.

Các mô hình NSFWJS thường dựa trên kiến trúc mạng tích chập nhẹ như `MobileNet` hoặc `MobileNetV2`. `MobileNetV2` sử dụng `depthwise separable convolution`, `inverted residual` và `linear bottleneck` để giảm số tham số và chi phí tính toán [6]. Thay vì áp dụng convolution đầy đủ trên cả chiều không gian và chiều kênh, `depthwise separable convolution` tách phép toán thành hai bước: `depthwise convolution` áp dụng bộ lọc riêng cho từng kênh, sau đó `pointwise convolution` 1×1 trộn thông tin giữa các kênh. Nếu kernel có kích thước $K \times K$, đầu vào có M kênh, đầu

ra có N kênh và feature map có kích thước $D_F \times D_F$, chi phí convolution chuẩn xấp xỉ:

$$K^2 \cdot M \cdot N \cdot D_F^2$$

Trong khi depthwise separable convolution có chi phí:

$$K^2 \cdot M \cdot D_F^2 + M \cdot N \cdot D_F^2$$

Tỷ lệ giảm chi phí xấp xỉ:

$$\frac{K^2 M D_F^2 + M N D_F^2}{K^2 M N D_F^2} = \frac{1}{N} + \frac{1}{K^2}$$

Với $K = 3$, mức giảm là đáng kể, giúp mô hình phù hợp với môi trường Node.js/TensorFlow.js CPU như trong hệ thống. Trong pipeline thực tế, ảnh được resize về 224×224 , loại bỏ alpha channel để tránh sai lệch shape tensor, chuyển thành tensor ba chiều $[H, W, 3]$, sau đó đưa vào `nsfwModel.classify`. Việc dispose tensor sau khi dự đoán cũng quan trọng vì worker xử lý nhiều ảnh liên tiếp; nếu không giải phóng tensor, bộ nhớ tiến trình Node.js có thể tăng dần và gây mất ổn định.

Về mặt hệ thống, NSFWJS không nên được xem là bộ phán quyết tuyệt đối. Mô hình học máy có thể tạo false positive hoặc false negative, đặc biệt với ảnh nghệ thuật, ảnh manga hoặc hình minh họa có phong cách phi thực. Vì vậy, thiết kế của hệ thống là dùng NSFWJS như bộ lọc tự động tăng đầu. Khi ảnh vượt ngưỡng, hệ thống không xóa ngay dữ liệu, mà tạo ảnh làm mờ, lưu `errorMetadata` gồm lý do, điểm số và ngưỡng, đồng thời chuyển tác phẩm sang `ACTION_REQUIRED`. Quy trình này cân bằng giữa tự động hóa và khả năng can thiệp của con người.

3.3.3 Thuật toán pHash và khoảng cách Hamming

Perceptual Hashing, hay băm nhận thức, là họ thuật toán sinh mã băm dựa trên đặc trưng cảm nhận của ảnh thay vì byte nhị phân. Mục tiêu của pHash là hai ảnh giống nhau về mặt thị giác sẽ có mã hash gần nhau, kể cả khi chúng khác kích thước, mức nén, định dạng hoặc có biến đổi nhỏ. Điều này khác căn bản với băm mật mã, nơi chỉ một thay đổi nhỏ trong tệp cũng làm hash thay đổi gần như hoàn toàn.

Một cách xây dựng pHash phổ biến dựa trên biến đổi cosine rời rạc, Discrete Cosine Transform (DCT). Quy trình tổng quát gồm các bước: chuẩn hóa ảnh về

kích thước cố định, chuyển ảnh sang thang xám, áp dụng DCT hai chiều, lấy vùng tần số thấp, tính ngưỡng trung bình hoặc trung vị, sau đó sinh chuỗi bit bằng cách so sánh từng hệ số với ngưỡng. DCT hai chiều của ma trận ảnh $f(x, y)$ kích thước $N \times N$ được định nghĩa:

$$C(u, v) = \alpha(u)\alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos \left[\frac{(2x+1)u\pi}{2N} \right] \cos \left[\frac{(2y+1)v\pi}{2N} \right] \quad (3.5)$$

Trong đó:

$$\alpha(0) = \sqrt{\frac{1}{N}}, \quad \alpha(k) = \sqrt{\frac{2}{N}}, \quad k > 0$$

Các hệ số DCT ở vùng chỉ số nhỏ biểu diễn thành phần tần số thấp, tương ứng với cấu trúc tổng thể, vùng sáng tối lớn và bố cục chính của ảnh. Các hệ số tần số cao thường chứa chi tiết nhỏ, nhiễu hoặc biến thiên sắc nét. Khi pHash giữ lại vùng tần số thấp, mã hash trở nên bền vững hơn trước nén JPEG, resize hoặc nhiễu. Giả sử chọn m hệ số DCT sau khi bỏ hệ số DC đầu tiên, đặt ngưỡng là trung vị μ , bit thứ i của hash được tính:

$$b_i = \begin{cases} 1, & C_i > \mu \\ 0, & C_i \leq \mu \end{cases}$$

Khi đó hash là vector bit $\mathbf{b} = (b_1, b_2, \dots, b_m)$. Để so sánh hai ảnh, hệ thống dùng khoảng cách Hamming giữa hai mã hash:

$$d_H(\mathbf{b}, \mathbf{b}') = \sum_{i=1}^m \mathbf{1}(b_i \neq b'_i) \quad (3.6)$$

Khoảng cách Hamming càng nhỏ thì hai ảnh càng giống nhau theo đặc trưng cảm nhận. Trong codebase, `DuplicateDetectionService` sinh pHash bằng `sharp-phash`, lấy tất cả pHash hiện có trong bảng `artwork_images`, tính khoảng cách bằng `sharp-phash/distance`, sau đó coi ảnh là trùng lặp nếu khoảng cách nhỏ hơn hoặc bằng ngưỡng. Phương thức `checkAndReject` sử dụng ngưỡng mặc định nhỏ, ví dụ 5, để phát hiện ảnh gần như giống hệt, còn phương thức tìm trùng lặp có thể dùng ngưỡng lớn hơn để hỗ trợ kiểm duyệt.

Ưu điểm của pHash là đơn giản, nhanh và không yêu cầu huấn luyện mô hình.

Tuy nhiên, pHash không giải quyết đầy đủ bài toán đạo nhái ở mức ý tưởng, ví dụ một tác phẩm được vẽ lại với bố cục tương tự nhưng thay đổi nét vẽ đáng kể. Vì vậy, trong hệ thống đề xuất, pHash được sử dụng như lớp phát hiện trùng lặp gần, còn tìm kiếm ngữ nghĩa bằng embedding có thể hỗ trợ phát hiện tương đồng mềm hơn trong các hướng mở rộng sau này.

3.3.4 Không gian vector, KNN và pgvector

Không gian vector đa chiều là nền tảng toán học của tìm kiếm ngữ nghĩa. Mỗi đối tượng như ảnh, văn bản hoặc phác thảo được ánh xạ thành một vector $\mathbf{x} \in \mathbb{R}^d$, trong đó d là số chiều embedding. Trong hệ thống, $d = 512$. Mỗi chiều không nhất thiết có diễn giải trực tiếp như một thuộc tính con người đặt tên, nhưng toàn bộ vector mã hóa một tổ hợp đặc trưng học được từ mô hình. Khi hai vector gần nhau theo một độ đo khoảng cách, hệ thống giả định hai đối tượng tương ứng cũng gần nhau về ngữ nghĩa hoặc thị giác.

Bài toán tìm kiếm lân cận gần nhất KNN được phát biểu như sau. Cho tập vector cơ sở dữ liệu:

$$\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}, \quad \mathbf{x}_i \in \mathbb{R}^d$$

và vector truy vấn $\mathbf{q} \in \mathbb{R}^d$, cần tìm tập K vector có khoảng cách nhỏ nhất tới $\mathbf{q}$:

$$\text{KNN}(\mathbf{q}, K) = \arg \min_{S \subset \mathcal{D}, |S|=K} \sum_{\mathbf{x} \in S} \text{dist}(\mathbf{q}, \mathbf{x}) \quad (3.7)$$

Với embedding CLIP, cosine distance thường được sử dụng:

$$\text{dist}_{\cos}(\mathbf{q}, \mathbf{x}) = 1 - \text{cosine}(\mathbf{q}, \mathbf{x})$$

Tìm KNN chính xác bằng cách quét tuyến tính toàn bộ n vector có độ phức tạp $O(nd)$ cho mỗi truy vấn. Khi n tăng lên hàng triệu ảnh, cách làm này trở nên tốn kém. Vì vậy, các hệ thống vector search thường dùng Approximate Nearest Neighbor (ANN), chấp nhận sai số nhỏ để đổi lấy tốc độ truy vấn cao hơn.

pgvector là extension của PostgreSQL cho phép lưu vector và truy vấn khoảng cách ngay trong cơ sở dữ liệu quan hệ [7]. pgvector hỗ trợ các toán tử khoảng cách như L2, inner product và cosine. Trong hệ thống, truy vấn sử dụng toán tử $<=>$ để tính cosine distance giữa embedding ảnh và embedding truy vấn. Câu truy vấn lấy ảnh gần nhất cho từng tác phẩm, sau đó sắp xếp theo similarity giảm dần:

```

1 WITH nearest_image_per_artwork AS (
2     SELECT DISTINCT ON (ai.artwork_id)
3         ai.artwork_id,
4         1 - (ai.embedding <=> $1::vector) AS similarity
5     FROM artwork_images ai
6     WHERE ai.embedding IS NOT NULL
7     ORDER BY ai.artwork_id, ai.embedding <=> $1::vector
8 )
9 SELECT artwork_id, similarity
10 FROM nearest_image_per_artwork
11 WHERE similarity > $3
12 ORDER BY similarity DESC
13 LIMIT $2
    
```

Listing 3.2: Truy vấn vector với pgvector

Điểm đáng chú ý là hệ thống tìm ảnh gần nhất ở cấp `ArtworkImage`, nhưng kết quả trả về ở cấp `Artwork`. Điều này phù hợp với mô hình dữ liệu nhiều ảnh: chỉ cần một ảnh trong tác phẩm tương đồng với truy vấn thì tác phẩm đó có thể đáng được hiển thị. Sau khi có danh sách `artwork_id` và `similarity`, service hydrate kết quả bằng Prisma để lấy tiêu đề, tác giả, thumbnail, tag và các bộ đếm tương tác.

Về chỉ mục, `pgvector` hỗ trợ HNSW và IVFFlat. HNSW, viết tắt của Hierarchical Navigable Small World, xây dựng đồ thị nhiều tầng trong đó mỗi vector là một nút và cạnh nối tới các nút lân cận [8]. Truy vấn bắt đầu từ tầng trên cùng, di chuyển tham lam tới nút gần truy vấn hơn, rồi đi xuống các tầng thấp hơn để tinh chỉnh kết quả. HNSW thường có tốc độ truy vấn cao và recall tốt, đổi lại chi phí bộ nhớ và thời gian xây dựng chỉ mục lớn hơn. IVFFlat chia không gian vector thành các cụm bằng thuật toán tương tự k-means; khi truy vấn, hệ thống chỉ quét một số cụm gần vector truy vấn thay vì toàn bộ dữ liệu. IVFFlat có chi phí lưu trữ thấp hơn nhưng cần chọn số lượng cụm và số cụm probe phù hợp.

Trong phạm vi đồ án, việc sử dụng `pgvector` có lợi thế kiến trúc: dữ liệu quan hệ và vector cùng nằm trong PostgreSQL, giảm nhu cầu vận hành thêm một vector database độc lập. Điều này phù hợp với modular monolith và quy mô ban đầu. Khi dữ liệu tăng rất lớn, hệ thống có thể cân nhắc tách vector search sang dịch vụ chuyên dụng, nhưng mô hình hiện tại giúp đơn giản hóa triển khai và vẫn cung cấp được năng lực tìm kiếm ngữ nghĩa cốt lõi.

3.4 Các dịch vụ trung gian và hạ tầng xử lý

3.4.1 Message Queue với BullMQ và Redis

Message Queue là cơ chế tách nơi phát sinh công việc khỏi nơi thực thi công việc. Trong hệ thống đề xuất, API đăng tải tác phẩm không trực tiếp kiểm duyệt và xử lý ảnh trong request. Thay vào đó, API tạo job và đưa vào hàng đợi BullMQ; worker nhận job và thực hiện xử lý nền. BullMQ sử dụng Redis làm backend lưu hàng đợi, trạng thái job, retry, progress và metadata [9]. Mô hình này giải quyết ba vấn đề: giảm thời gian phản hồi của API, cô lập lỗi xử lý ảnh khỏi luồng tương tác chính và cho phép theo dõi tiến độ từng job.

Luồng xử lý ảnh có thể mô tả theo trình tự: người dùng tải ảnh lên, backend tạo bản ghi Artwork ở trạng thái PROCESSING, lưu file tạm vào storage, đẩy ProcessArtworkJob vào queue, worker tải file, tính pHash, kiểm tra NSFW, sinh biến thể ảnh, ghi ArtworkImage, cập nhật trạng thái Artwork, lập chỉ mục Meilisearch và sinh embedding. Mỗi bước có thể cập nhật progress để frontend hiển thị trạng thái xử lý cho người dùng. Nếu một ảnh bị trùng lặp hoặc NSFW, worker không nhất thiết dừng toàn bộ job, mà ghi kết quả lỗi ở cấp ảnh và tiếp tục xử lý các ảnh còn lại.

Redis trong hệ thống không chỉ phục vụ BullMQ. Redis còn được dùng làm cache cho vector embedding truy vấn văn bản trong AiSearchService; khóa cache có dạng gắn với phiên bản embedding để tránh dùng nhầm vector cũ khi thay mô hình. Redis cũng được dùng để chống spam lượt xem bằng cách đặt key debounce theo người dùng hoặc IP trong một khoảng thời gian. Ngoài ra, MeilisearchSyncService dùng Redis Hash như buffer cập nhật chỉ số, gom nhiều thay đổi nhỏ trước khi flush sang Meilisearch. Điều này cho thấy Redis đóng vai trò hạ tầng tốc độ cao cho cả hàng đợi, cache, rate limit và buffer.

Mặc dù Redis hỗ trợ cơ chế publish-subscribe, mô hình BullMQ trong hệ thống gần với producer-consumer queue hơn là pub-sub thuần túy. Producer phát sinh job, Redis lưu job bền vững hơn so với message tạm thời, worker tiêu thụ job và cập nhật trạng thái. So với pub-sub thuần, queue phù hợp hơn với xử lý ảnh vì mỗi job cần được xử lý ít nhất một lần, có retry, có trạng thái thất bại và có thể quan sát tiến độ.

3.4.2 Meilisearch và inverted index

Meilisearch là công cụ tìm kiếm toàn văn nhẹ, tốc độ cao, hỗ trợ typo tolerance, ranking rule, filter, sort và API đơn giản [10]. Trong hệ thống, Meilisearch đảm nhiệm lớp lexical search cho tác phẩm. Mỗi document trong chỉ mục artworks gồm các trường như id, title, description, author, thumb-

nail, tags, rating, isAI, createdAt, likeCount, viewCount, ratioClass, maxResolution và isHighRes. Các thuộc tính searchable gồm tiêu đề, tag, tên hiển thị của tác giả và mô tả; các thuộc tính filterable gồm tag, rating, AI flag, tác giả, tỷ lệ ảnh và độ phân giải; các thuộc tính sortable gồm thời gian tạo, lượt thích và lượt xem.

Nền tảng lý thuyết của tìm kiếm toàn văn là inverted index. Thay vì lưu mỗi document rồi quét tất cả document khi có truy vấn, search engine xây dựng ánh xạ từ term sang danh sách document chứa term đó. Nếu gọi D là tập document và V là tập token, inverted index lưu:

$$\text{postings}(w) = \{d \in D \mid w \in d\}$$

Khi người dùng tìm kiếm một từ hoặc cụm từ, hệ thống truy xuất postings list tương ứng, kết hợp chúng theo toán tử truy vấn, sau đó xếp hạng kết quả dựa trên các quy tắc như mức độ khớp từ, khoảng cách từ, thuộc tính chứa từ, độ chính xác và chỉ số phổ biến. Meiliseach bổ sung typo tolerance để xử lý lỗi gõ, điều rất hữu ích trong tìm kiếm tag hoặc tên tác phẩm.

Trong hệ thống, Meiliseach không phải nguồn dữ liệu gốc. PostgreSQL vẫn là source of truth; Meiliseach là chỉ mục phụ để tối ưu tìm kiếm. Vì vậy, sau khi một tác phẩm được xử lý thành công, worker lập chỉ mục document tương ứng. Khi bộ đếm lượt xem hoặc lượt thích thay đổi, hệ thống không cập nhật Meiliseach ngay từng sự kiện nhỏ, mà gom cập nhật qua Redis buffer và flush theo lô. Cách làm này giảm số request tới Meiliseach và tránh gây nghẽn khi tác phẩm có nhiều tương tác.

3.4.3 Clerk Auth

Clerk là dịch vụ xác thực và quản lý người dùng cung cấp đăng nhập, đăng ký, session, token và SDK cho frontend/backend [11]. Trong hệ thống, Clerk xử lý phần định danh bên ngoài, còn backend quản lý hồ sơ nghiệp vụ nội bộ. Frontend được bao bọc bởi ClerkProvider; middleware bảo vệ các route như dashboard và admin; backend dùng ClerkStrategy để xác minh Bearer token. Khi token hợp lệ, backend lấy sub làm clerkId, gọi Clerk API lấy email, username và avatar, sau đó gọi UsersService.findOrCreateByClerkId.

Cách thiết kế lazy sync này có ưu điểm là người dùng chỉ được tạo trong database nội bộ khi thực sự truy cập backend. Bản ghi nội bộ sau đó lưu thêm những thuộc tính mà Clerk không quyết định, ví dụ vai trò quản trị, trạng thái artist, cảnh báo, ban, sở thích, stripeCustomerId và các quan hệ xã hội. Tách xác thực khỏi

phân quyền nghiệp vụ là nguyên tắc quan trọng: Clerk trả lời câu hỏi “người này là ai”, còn backend trả lời câu hỏi “người này được phép làm gì trong nền tảng”.

3.4.4 Stripe và quản lý thanh toán

Stripe là cổng thanh toán cung cấp checkout, subscription, invoice, customer portal và webhook [12]. Trong hệ thống đề xuất, Stripe được dùng cho hai nhóm nghiệp vụ: subscription nền tảng và subscription theo artist tier. Backend tạo checkout session với metadata chứa `userId`, `planId`, `tierId` hoặc `artistId`; khi thanh toán hoàn tất, webhook Stripe gửi sự kiện về backend để đồng bộ trạng thái. Các sự kiện như `checkout.session.completed`, `invoice.paid`, `customer.subscription.updated` và `customer.subscription.deleted` được xử lý để cập nhật subscription nội bộ.

Một điểm quan trọng trong tích hợp thanh toán là không tin cậy hoàn toàn vào redirect phía frontend. Sau khi người dùng thanh toán, trình duyệt có thể bị đóng, mạng có thể lỗi hoặc người dùng không quay lại trang success. Vì vậy, webhook mới là cơ chế đồng bộ đáng tin cậy. Backend dùng chữ ký `stripe-signature` để xác minh payload webhook, sau đó ánh xạ dữ liệu Stripe sang bảng nội bộ. Với mô hình artist tier, subscription không chỉ biểu diễn quan hệ người dùng với nền tảng, mà còn biểu diễn quan hệ người hâm mộ với một artist cụ thể. Điều này cho phép backend kiểm tra quyền xem nội dung `TIER_GATED` dựa trên dữ liệu subscription đã đồng bộ.

Từ góc độ kiến trúc, Stripe được đóng gói trong `StripeService`, còn logic nghiệp vụ nằm trong `PaymentsService`. `StripeService` chịu trách nhiệm gọi API Stripe như tạo checkout session, lấy subscription, tạo customer, tạo billing portal và xác minh webhook. `PaymentsService` chịu trách nhiệm kiểm tra người dùng, tạo plan hoặc tier, đảm bảo customer tồn tại, cập nhật database và áp dụng quy tắc nghiệp vụ như không cho đổi giá tier khi đã có subscriber đang hoạt động. Sự phân tách này giúp hệ thống không để logic nghiệp vụ phụ thuộc trực tiếp vào API thanh toán ở mọi nơi.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

4.1 Thiết kế kiến trúc hệ thống

4.1.1 Lựa chọn kiến trúc phần mềm

4.1.2 Kiến trúc tổng thể

4.1.3 Thiết kế các module backend

4.1.4 Thiết kế các phân vùng frontend

4.1.5 Thiết kế luồng xử lý nền

4.2 Thiết kế cơ sở dữ liệu

4.2.1 Mô hình thực thể quan hệ

4.2.2 Thiết kế bảng người dùng và phân quyền

4.2.3 Thiết kế bảng tác phẩm và ảnh

4.2.4 Thiết kế bảng tương tác xã hội

4.2.5 Thiết kế bảng thanh toán và gói thành viên

4.3 Thiết kế API và giao tiếp hệ thống

4.3.1 API xác thực và người dùng

4.3.2 API quản lý tác phẩm

4.3.3 API tìm kiếm và khám phá

4.3.4 API quản trị và kiểm duyệt

4.3.5 API thanh toán và webhook

4.4 Triển khai hệ thống

4.4.1 Môi trường phát triển

4.4.2 Cấu hình cơ sở dữ liệu và dịch vụ trung gian

4.4.3 Triển khai frontend

4.4.4 Triển khai backend

4.4.5 Triển khai worker xử lý nền

4.5 Đánh giá và kiểm thử

4.5.1 Kiểm thử chức năng

4.5.2 Kiểm thử tích hợp

4.5.3 Đánh giá hiệu năng xử lý ảnh

4.5.4 Đánh giá tìm kiếm

4.5.5 Đánh giá bảo mật và phân quyền

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

5.1 Tự động hóa pipeline xử lý ảnh

5.1.1 Tách tác vụ nặng khỏi luồng request

5.1.2 Quản lý tiến độ và trạng thái xử lý

5.1.3 Xử lý lỗi cục bộ ở cấp ảnh

5.2 Kiểm duyệt nội dung bán tự động

5.2.1 Phân loại NSFW trong worker

5.2.2 Lưu metadata phục vụ điều phối

5.2.3 Cân bằng giữa tự động hóa và kiểm duyệt thủ công

5.3 Phát hiện ảnh trùng lặp bằng pHash

5.3.1 Sinh dấu vân tay nhận thức

5.3.2 So sánh bằng khoảng cách Hamming

5.3.3 Ứng dụng trong bảo vệ nội dung

5.4 Tìm kiếm lai toàn văn và ngữ nghĩa

5.4.1 Lập chỉ mục Meilisearch

5.4.2 Lưu trữ vector bằng pgvector

5.4.3 Kết hợp kết quả tìm kiếm trong trải nghiệm người dùng

5.5 Mô hình thành viên cho người sáng tạo

5.5.1 Thiết kế artist tier

5.5.2 Tích hợp Stripe subscription

5.5.3 Kiểm soát quyền truy cập nội dung

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

6.1.1 Kết quả đạt được

6.1.2 Mức độ đáp ứng mục tiêu đề tài

6.1.3 Các hạn chế còn tồn tại

6.2 Hướng phát triển

6.2.1 Mở rộng khả năng kiểm duyệt

6.2.2 Tối ưu tìm kiếm vector

6.2.3 Cải tiến hệ thống khuyến nghị

6.2.4 Mở rộng cơ chế bản quyền và khiếu nại

6.2.5 Tối ưu triển khai hạ tầng quy mô lớn

TÀI LIỆU THAM KHẢO

- [1] NestJS, *Nestjs documentation*, 2026. [Online]. Available: <https://docs.nestjs.com/>
- [2] Prisma, *Prisma documentation*, 2026. [Online]. Available: <https://www.prisma.io/docs>
- [3] The PostgreSQL Global Development Group, *Postgresql documentation*, 2026. [Online]. Available: <https://www.postgresql.org/docs/>
- [4] Vercel, *Next.js documentation*, 2026. [Online]. Available: <https://nextjs.org/docs>
- [5] A. Radford et al., “Learning transferable visual models from natural language supervision,” in *Proceedings of the 38th International Conference on Machine Learning*, 2021.
- [6] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “Mobilenetv2: Inverted residuals and linear bottlenecks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018.
- [7] pgvector, *Pgvector documentation*, 2026. [Online]. Available: <https://github.com/pgvector/pgvector>
- [8] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2018.
- [9] Taskforce.sh, *Bullmq documentation*, 2026. [Online]. Available: <https://docs.bullmq.io/>
- [10] Meilisearch, *Meilisearch documentation*, 2026. [Online]. Available: <https://www.meilisearch.com/docs>
- [11] Clerk, *Clerk documentation*, 2026. [Online]. Available: <https://clerk.com/docs>
- [12] Stripe, *Stripe documentation*, 2026. [Online]. Available: <https://docs.stripe.com/>